

**A MINI PROJECT REPORT
ON
"STOCK PORTFOLIO ANALYZER"**

Submitted by

SATVIK GUPTA

University Roll No.: 2400950100083

In partial fulfilment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Mahatma Gandhi Mission's College of Engineering & Technology,
Noida**

August, 2026

SUBJECT CODE: MINI PROJECT

CERTIFICATE

This is to certify that the Mini Project Report entitled “STOCK PORTFOLIO ANALYZER” is a bona fide record of the work carried out by **SATVIK GUPTA** (University Roll No. 2400950100083), a student of the Third Year B.Tech programme in the Department of Computer Science & Engineering, Mahatma Gandhi Mission's College of Engineering and Technology, Noida, during the period 25th June 2026 to 5th August 2026, under supervision and guidance, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering.

The matter embodied in this report has not been submitted, in part or in full, to any other University or Institute for the award of any other degree or diploma.

Date: _____

Place: Noida

Internal Guide

Head of Department

DECLARATION

I, **SATVIK GUPTA**, hereby declare that the Mini Project Report entitled “STOCK PORTFOLIO ANALYZER”, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Mahatma Gandhi Mission's College of Engineering and Technology, Noida, is an authentic and original record of the work carried out by me during the period 25th June 2026 to 5th August 2026.

I further declare that this report has not been submitted earlier for the award of any degree or diploma of this or any other Institute, and that all sources of information used have been duly acknowledged.

Date: _____

Place: Noida

Signature of the Candidate

SATVIK GUPTA

University Roll No.: 2400950100083

ACKNOWLEDGEMENT

I express my sincere gratitude to the Department of Computer Science & Engineering, Mahatma Gandhi Mission's College of Engineering and Technology, Noida, for providing the opportunity to undertake this mini project. I am thankful to my Internal Guide and the Head of the Department for their valuable guidance and continuous encouragement throughout the project period (25th June 2026 – 5th August 2026).

I also acknowledge the open-source communities behind React.js, Spring Boot, Hibernate, MySQL, JWT, BCrypt, and related tools that made the development of this full-stack application possible.

SATVIK GUPTA

ABSTRACT

Manual management of stock investments through notebooks or spreadsheets is time-consuming, error-prone, and lacks the ability to provide investors with a real-time, consolidated view of their financial holdings. The Stock Portfolio Analyzer is a full-stack, three-tier web application designed to address these shortcomings by offering investors a centralized, digital platform to securely register, track, and analyze their stock market investments.

The system is built using React.js on the presentation layer, Spring Boot with Spring MVC and Spring Data JPA on the business logic layer, and MySQL as the persistent relational data store, with all inter-tier communication realized through a protected RESTful API. A key feature of the final implementation is the robust security architecture utilizing JSON Web Tokens (JWT) for stateless authentication and BCrypt for secure password hashing. The application supports secure user registration, login, profile management, maintenance of a master list of tradable stocks, execution of buy and sell transactions, and computation of running portfolio holdings. Furthermore, it strictly enforces authenticated user ownership, ensuring data privacy and integrity across all operations.

The backend follows a layered Controller–Service–Repository–Entity design, leveraging Hibernate as the Object-Relational Mapping (ORM) provider. The REST APIs include fully protected endpoints for personalized dashboards, transaction histories, and data exports (CSV and Excel). The resulting system demonstrates the practical application of modern full-stack development principles to a real-world personal finance problem. This report discusses the system’s architecture, database design, API specifications, and testing outcomes, alongside its advantages, limitations, and scope for future enhancement.

Keywords: Stock Portfolio Management, Full-Stack Web Application, Spring Boot, React.js, REST API, MySQL, JWT Authentication, BCrypt, Three-Tier Architecture, Hibernate ORM.

TABLE OF CONTENTS

(Table of Contents will be generated by the PDF renderer)

LIST OF FIGURES

Figure 4.1: Three-Tier Architecture of the System

Figure 4.2: Controller–Service–Repository–Entity Flow

Figure 4.3: JWT Authentication Architecture Flow

Figure 5.1: Entity Relationship Diagram

Figure 8.1: Login Interface

Figure 8.2: User Signup Interface

Figure 8.3: Portfolio Dashboard

Figure 8.4: Holdings Interface

Figure 8.5: Stock Explorer

Figure 8.6: Transaction History

Figure 8.7: User Profile and Portfolio Summary

Figure 8.8: Change Password Interface

LIST OF TABLES

Table 3.1: Frontend Technology Stack
Table 3.2: Backend Technology Stack
Table 3.3: Database and Testing Tools
Table 5.1: Structure of the users Table
Table 5.2: Structure of the stock Table
Table 5.3: Structure of the transaction Table
Table 7.1: Authentication APIs
Table 7.2: Profile and Export APIs
Table 7.3: Stock and Transaction APIs
Table 9.1: Comprehensive Test Cases and Results
Table 10.1: Advantages of the Proposed System
Table 10.2: Limitations of the Proposed System
Table A.1: List of Abbreviations

CHAPTER 1: INTRODUCTION

1.1 Overview

Investing in the stock market has become increasingly accessible, yet effectively tracking and analyzing a personal investment portfolio remains a challenge for many retail investors who still rely on notebooks or generic spreadsheets. The Stock Portfolio Analyzer is a full-stack web application that provides a dedicated, centralized digital platform to securely register users, maintain a catalogue of stocks, record buy and sell transactions, and obtain a consolidated view of portfolio composition and value.

The application follows a three-tier architecture separating the presentation layer (React.js), the business logic layer (Spring Boot), and the data persistence layer (MySQL), communicating through a well-defined and protected REST API. A core component of the modern system is its security infrastructure, relying on JSON Web Tokens (JWT) and BCrypt password hashing to ensure strict data privacy and user ownership. This report documents the complete design and implementation of the system.

1.2 Problem Statement

Managing a stock portfolio manually using notebooks or spreadsheets is time-consuming and error-prone for several reasons:

- **Manual Portfolio Tracking:** Manual entry of transactions is tedious and prone to calculation errors as transaction volume grows.
- **Fragmented Transaction Records:** There is no structured, easily queryable transaction history linking a specific user, stock, and the action performed over time.
- **Calculation Difficulty:** Computing running portfolio value, average purchase price, and realized/unrealized profit/loss requires fragile, manually maintained formulas.
- **Lack of Secure Authentication:** Spreadsheets provide no secure authentication, leaving personal financial data vulnerable.
- **Lack of Centralized Visibility:** Investors struggle to get a unified snapshot of their net worth across different equity holdings.
- **Difficulty Maintaining Personal Records:** Without a centralized database, maintaining historical data across devices is challenging.
- **Lack of Convenient Data Export:** Consolidating data for tax purposes or external analysis is cumbersome without automated CSV or Excel export features.

The proposed Stock Portfolio Analyzer solves these problems through a centralized, rule-enforcing digital platform backed by a relational database and accessed via a responsive, highly secure web interface.

1.3 Objectives of the Project

The primary objectives of the Stock Portfolio Analyzer project are to:

- Provide a digital, centralized platform for personalized portfolio management, eliminating manual record-keeping.
- Implement secure authentication using JWT and robust password security using BCrypt hashing.
- Support reliable Create, Read, Update, and Delete (CRUD) operations for users, stocks, and transactions via protected REST APIs.
- Ensure consistent data management and relational integrity using a well-normalized MySQL database.
- Automate complex holdings calculations, including average buy price, invested value, and real-time profit/loss.
- Deliver personalized profile management features, including secure password changes and profile updates.
- Enable convenient data export functionality, allowing users to download their transaction history (CSV) and portfolio summary (Excel).

- Design a simple, responsive user interface requiring minimal learning effort.
- Build on a scalable, layered (Controller–Service–Repository–Entity) architecture for future extensibility.

1.4 Scope of the Project

The current implementation covers secure user registration and login, a master stock catalogue, buy/sell transaction recording, real-time computation of holdings and investment value, profile management, and comprehensive data export capabilities (CSV and Excel). A REST API layer exposes these capabilities to the React frontend, tightly guarded by JWT authorization ensuring strict user data isolation.

Features such as live stock price feeds, AI-based investment recommendations, advanced visual analytics with charts, automated email alerts, and a dedicated mobile application are identified as future scope and are outside the current implementation, which focuses on establishing a robust, secure, and accurate core transaction engine.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

Most retail investors who do not use dedicated portfolio software rely on manual notebooks, generic spreadsheets with self-designed formulas, or the fragmented transaction history provided by individual brokerage platforms. These approaches lack data validation, do not scale well as transaction volume grows, offer no structured way to query historical data, and enforce no business rules.

2.2 Problems in Existing System

The existing manual methods present several significant challenges:

- Spreadsheets enforce no referential integrity, allowing inconsistent entries such as selling shares that were never purchased.
- Manual calculations for average cost basis and profit/loss are highly susceptible to human error.
- Data privacy is virtually non-existent, as files are easily shared or misplaced without cryptographic protection.
- Aggregating historical transactions for reporting or tax filing requires manual extraction and formatting.
- Multi-device synchronization is often difficult and prone to version conflicts.

2.3 Proposed System

The proposed Stock Portfolio Analyzer overcomes these limitations by introducing a dedicated three-tier web application with a relational database and comprehensive security model at its core, offering:

- A structured relational schema enforcing referential integrity via primary and foreign keys.
- A stateless, JWT-protected RESTful API layer exposing well-defined endpoints.
- A dedicated Service layer that centralizes business logic, such as portfolio valuation, sell-transaction validation, and BCrypt password verification.
- A responsive, component-based React.js frontend with a personalized dashboard, stock explorer, transaction history, and detailed profile page.
- Automated CSV and Excel export generation for seamless data portability.

2.4 Advantages of Proposed Approach

The integration of JWT authentication, BCrypt, and authenticated ownership represents a massive improvement over basic user-management approaches. By identifying the user exclusively through a cryptographically signed token rather than trusting a frontend-supplied ID, the system completely eliminates Insecure Direct Object Reference (IDOR) vulnerabilities. Users can only access their own profile, transactions, and holdings, guaranteeing privacy. Furthermore, the automated calculation engine removes human error from portfolio valuation.

2.5 Feasibility Study

2.5.1 Technical Feasibility

The chosen technologies — React.js, Spring Boot, Spring Security, Spring Data JPA, Hibernate, and MySQL — are mature, well-documented, open-source tools that run efficiently on standard development machines. The project's technical requirements are well within the capabilities of a Third-Year B.Tech mini project.

2.5.2 Economic Feasibility

Every tool used in the development of the Stock Portfolio Analyzer is free and open-source. This includes React.js, Spring Boot, MySQL, Hibernate, Maven, IntelliJ IDEA, VS Code, and Bruno. Consequently, the project incurs zero recurring licensing costs, making it highly economically feasible.

2.5.3 Operational Feasibility

The simple, intuitive user interface allows a user with only basic computer literacy to securely register, add stocks, record transactions, and view their personalized portfolio dashboard without extensive training. The modular REST-based backend is straightforward to maintain, test, and extend in future iterations.

CHAPTER 3: TECHNOLOGY STACK AND TOOLS

The Stock Portfolio Analyzer has been developed using modern, industry-standard technologies chosen to provide a robust, secure foundation for a three-tier web application.

3.1 Frontend Technologies

Technology	Purpose
React.js	JavaScript library used to build a component-based, single-page user interface.
React Router	Library used for declarative routing and protected route enforcement.
HTML5 / CSS3	Used for structuring semantic content and implementing a responsive styling layout.
JavaScript (ES6+)	Core scripting language used for component logic and state management.
Axios	Promise-based HTTP client used to consume REST APIs, configured with interceptors to automatically attach JWT Bearer tokens.

Table 3.1: Frontend Technology Stack

3.2 Backend Technologies

Technology	Purpose
Java (JDK 17+)	Primary programming language used to implement the backend application.
Spring Boot (4.0.7)	Framework used to rapidly build a stand-alone, production-ready backend.
Spring Security 6	Provides robust authentication and authorization, handling the JWT filter chain.
jjwt (io.jsonwebtoken)	Library used for generating, signing, and parsing JSON Web Tokens.
BCryptPasswordEncoder	Cryptographic hashing algorithm used to securely store user passwords.
Spring Data JPA	Simplifies data access by providing repository abstractions over the persistence provider.
Hibernate	Object-Relational Mapping (ORM) framework mapping Java entities to MySQL.
Apache POI (5.2.3)	Java library used for generating dynamic Excel (XLSX) portfolio export reports.
Maven	Build automation and dependency management tool.

Table 3.2: Backend Technology Stack

3.3 Database and Development Tools

Tool	Purpose
MySQL	Relational database management system (RDBMS) used for all persistent data storage.
VS Code	Lightweight editor used for frontend (React.js) development.
IntelliJ IDEA	IDE used for backend (Java/Spring Boot) development.
Bruno / curl	Tools used to manually construct and test REST endpoints during development.

Table 3.3: Database and Testing Tools

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 Three-Tier Architecture

The Stock Portfolio Analyzer follows a classic three-tier architecture, separating the application into logically distinct layers: Presentation, Business Logic, and Database. This separation of concerns significantly improves maintainability, scalability, and security.

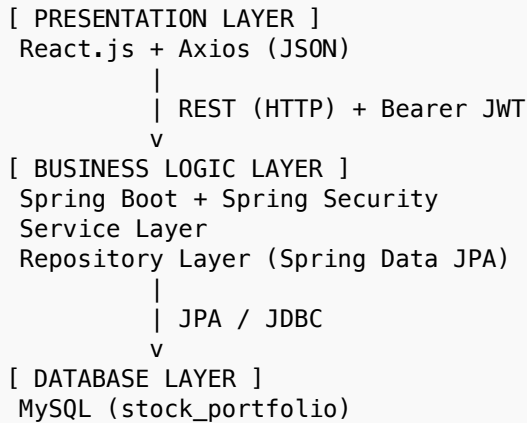


Figure 4.1: Three-Tier Architecture of the System

4.2 Controller-Service-Repository-Entity Architecture

Within the Spring Boot backend, the application utilizes a layered design pattern:

- **Controller:** Annotated with `@RestController`, these classes expose REST endpoints, parse incoming JSON HTTP requests, and delegate processing to the Service layer.
- **Service:** Contains the core business logic. It handles tasks such as portfolio valuation, transaction validation, BCrypt password verification, and Excel/CSV generation.
- **Repository:** Interfaces extending `JpaRepository` that provide built-in and custom CRUD methods for database access without writing boilerplate SQL.
- **Entity:** Plain Old Java Objects (POJOs) annotated with `@Entity`, mapped directly by Hibernate to the corresponding MySQL database tables.

4.3 Authentication Architecture

The system employs a stateless, token-based authentication mechanism using JSON Web Tokens (JWT) and BCrypt.

Signup Flow: When a user registers, their plaintext password is intercepted by the `AuthService`, hashed using `BCryptPasswordEncoder`, and stored securely in the database. The system immediately generates and returns a JWT.

Login Flow: The user submits their email and password. Spring Security retrieves the user record and verifies the password hash. If successful, a JWT is generated, signed with a secret key, and returned to the frontend. The frontend stores this token in `sessionStorage`.



Figure 4.3: JWT Authentication Architecture Flow

4.4 Request Lifecycle

For protected requests (e.g., viewing the dashboard), the lifecycle is as follows:

1. **React Frontend:** Axios interceptor attaches the JWT to the `Authorization: Bearer <token>` header.
2. **Security Filter:** `JwtAuthenticationFilter` intercepts the request, parses the token, validates the signature and expiration, and extracts the user email.
3. **Security Context:** The user's authentication token is placed in the `SecurityContextHolder`.
4. **Controller & Service:** The request reaches the Controller. The Service layer uses `CurrentUserService.getCurrentUser()` to retrieve the exact user entity making the request.
5. **Repository & DB:** The Repository queries the database using the authenticated user's ID, ensuring they only retrieve their own data.
6. **Response:** The data is returned as JSON to the frontend.

4.5 User Ownership Model

A critical architectural decision in the final implementation is the strict enforcement of user ownership. The backend **never** trusts a `userId` supplied by the frontend in a request payload or URL parameter for protected operations. Instead, the identity is always dynamically extracted from the cryptographically verified JWT in the Spring Security context. This guarantees that User A cannot manipulate a request payload to maliciously access or modify User B's portfolio, holdings, or profile, providing a robust defense against IDOR vulnerabilities.

CHAPTER 5: DATABASE DESIGN

5.1 Database Overview

The application uses a relational database named `stock_portfolio`, implemented in MySQL. The schema is designed according to normalization principles to eliminate redundancy and preserve data integrity, consisting of four principal tables: `users`, `stock`, `transactions`, and `watchlists`.

5.2 ER Diagram (Textual Representation)

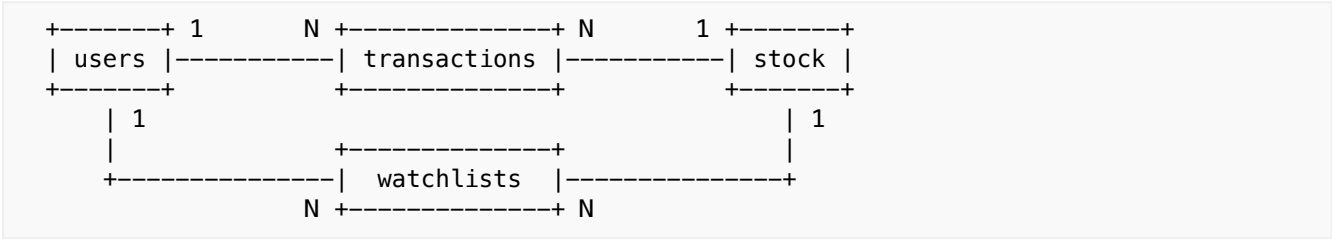


Figure 5.1: Entity Relationship Diagram

5.3 Users Table

Field	Data Type	Constraint	Description
id	BIGINT	PK, Auto Increment	Unique identifier for each user.
name	VARCHAR(255)	NOT NULL	Full name of the registered user.
email	VARCHAR(255)	NOT NULL, UNIQUE	Email address used for login.
password	VARCHAR(255)	NOT NULL	BCrypt hashed password.

Table 5.1: Structure of the users Table

5.4 Stock Table

Field	Data Type	Constraint	Description
id	BIGINT	PK, Auto Increment	Unique identifier for each stock.
symbol	VARCHAR(255)	NOT NULL, UNIQUE	Stock ticker symbol (e.g., TCS).
companyname	VARCHAR(255)	NOT NULL	Full registered name of the company.
sector	VARCHAR(255)	NULLABLE	Industry sector of the company.
currentprice	DOUBLE	NOT NULL	Latest recorded market price.

Table 5.2: Structure of the stock Table

5.5 Transaction Table

Field	Data Type	Constraint	Description
id	BIGINT	PK, Auto Increment	Unique identifier for each transaction.
user_id	BIGINT	FK → users(id)	References the user who performed the transaction.
stock_id	BIGINT	FK → stock(id)	References the stock involved.
quantity	INT	NOT NULL	Number of shares bought or sold.
price	DOUBLE	NOT NULL	Price per share at transaction time.
transaction_type	VARCHAR(255)	NOT NULL (BUY/SELL)	Indicates purchase or sale.
transaction_date	DATETIME(6)	NOT NULL	Date and time of the transaction.

Table 5.3: Structure of the transaction Table

5.6 Data Integrity and Security

Data integrity is enforced at the database level using Primary Keys (PK) and Foreign Keys (FK). The `user_id` and `stock_id` columns in the `transactions` and `watchlists` tables enforce strict referential integrity. Crucially, the `password` column in the `users` table is sized to accommodate a 60-character BCrypt hash. Plaintext passwords are never stored in the database, ensuring that even in the event of a database compromise, user credentials remain secure.

CHAPTER 6: MODULE DESCRIPTION

6.1 Authentication Module

The Authentication module manages secure access to the platform. It handles user registration (Signup) and session initiation (Login). The frontend collects credentials and stores the returned JWT. The backend verifies credentials, generates JWTs, and hashes passwords using BCrypt. Security is paramount here, ensuring passwords are never exposed in API responses.

6.2 Profile Module

The Profile module serves as the personal account center for the authenticated user. It displays personal information (Name, Email), a consolidated portfolio summary (Total Invested, Current Value, Profit/Loss), and allows the user to update their details or securely change their password. It also handles the orchestration of user logout by clearing the local session.

6.3 Stock Management Module

This module maintains the master catalogue of tradable stocks. It allows the retrieval of stock details, including symbols, company names, and current prices. This module provides the foundational market data required by the transaction and portfolio modules.

6.4 Transaction Management Module

The operational core of the system. It records every BUY and SELL action. The frontend provides a seamless interface for trade entry, while the backend rigorously validates transactions—most notably ensuring that a user cannot SELL more shares than they currently own (preventing negative holdings). It maintains a complete chronological history linked to the specific authenticated user.

6.5 Portfolio & Holdings Module

This module dynamically aggregates the user's transaction history to compute their current holdings. For each stock, it calculates the net quantity held, average purchase price, total invested value, current market value, and real-time profit/loss percentage. All calculations are performed on the backend to ensure consistency.

6.6 Dashboard Module

The Dashboard module provides a high-level, centralized overview of the user's financial standing. It aggregates data from the Portfolio module to present headline metrics: overall Total Invested, overall Current Value, and net Profit/Loss across all equities, alongside a snapshot of recent activity.

6.7 Data Export Module

A specialized module designed for data portability. It allows the authenticated user to download their financial records. The frontend handles Blob data conversion, while the backend utilizes Apache POI to generate comprehensive Excel workbooks (spanning multiple sheets for summaries, holdings, and transactions) and dynamic CSV files for transaction logs.

CHAPTER 7: SECURITY AND REST API DESIGN

7.1 REST API Principles

The backend exposes functionality through a stateless RESTful API. Resources are named using nouns, HTTP methods indicate the intended action, and standard HTTP status codes communicate outcomes. Request and response bodies are strictly encoded in JSON.

7.2 Authentication APIs (Public)

These endpoints are open and do not require a JWT.

Endpoint	Method	Auth	Purpose	Response
/api/auth/signup	POST	None	Registers a new user, hashes password via BCrypt.	200 OK + JWT
/api/auth/login	POST	None	Authenticates credentials and generates session token.	200 OK + JWT / 401

Table 7.1: Authentication APIs

7.3 Profile and Export APIs (Protected)

These endpoints require a valid JWT. They implicitly operate on the authenticated user derived from the token.

Endpoint	Method	Auth	Purpose	Response
/api/users/me	GET	JWT	Retrieves current user's profile details.	200 OK
/api/users/me	PUT	JWT	Updates current user's name or email.	200 OK
/api/users/me/password	PUT	JWT	Securely changes user's password.	200 OK / 401
/api/users/me/export/transactions/csv	GET	JWT	Generates a CSV of the user's transactions.	200 OK (text/csv)
/api/users/me/export/portfolio/excel	GET	JWT	Generates a multi-sheet Excel portfolio report.	200 OK (XLSX)

Table 7.2: Profile and Export APIs

7.4 Stock, Transaction, and Portfolio APIs (Protected)

Endpoint	Method	Auth	Purpose	Response
/api/dashboard	GET	JWT	Retrieves aggregated dashboard metrics.	200 OK
/api/portfolio	GET	JWT	Computes and retrieves current stock holdings.	200 OK
/api/transactions	GET	JWT	Retrieves full chronological transaction history.	200 OK
/api/transactions	POST	JWT	Records a new BUY or SELL transaction.	200 OK / 400
/api/stocks	GET	JWT	Retrieves master catalogue of available stocks.	200 OK
/api/watchlist	GET	JWT	Retrieves the user's saved watchlist.	200 OK
/api/watchlist	POST	JWT	Adds a stock to the user's watchlist.	200 OK

Table 7.3: Stock and Transaction APIs

7.5 Error Handling and Status Codes

The application strictly adheres to HTTP standards: **200 OK** for successful operations, **400 Bad Request** for invalid logic (e.g., selling unowned shares), **401 Unauthorized** for missing or invalid JWTs or incorrect current passwords, and **404 Not Found** for non-existent resources. Spring Boot's global exception handling intercepts errors to return clean, structured JSON error responses rather than stack traces.

CHAPTER 8: FRONTEND IMPLEMENTATION AND USER INTERFACE

This chapter presents the frontend user interface of the Stock Portfolio Analyzer. Implemented using React.js, the interface is designed to provide a seamless, secure experience. All screenshots are taken from the active, running application.

8.1 Login and Signup Pages

The authentication gateway to the application. The Signup page captures the user's name, email, and password. The Login page verifies these credentials. Upon successful authentication, the frontend receives a JWT, stores it securely, and redirects the user to the Dashboard. Unauthenticated attempts to access any other route automatically redirect back to the Login page.

8.2 Dashboard

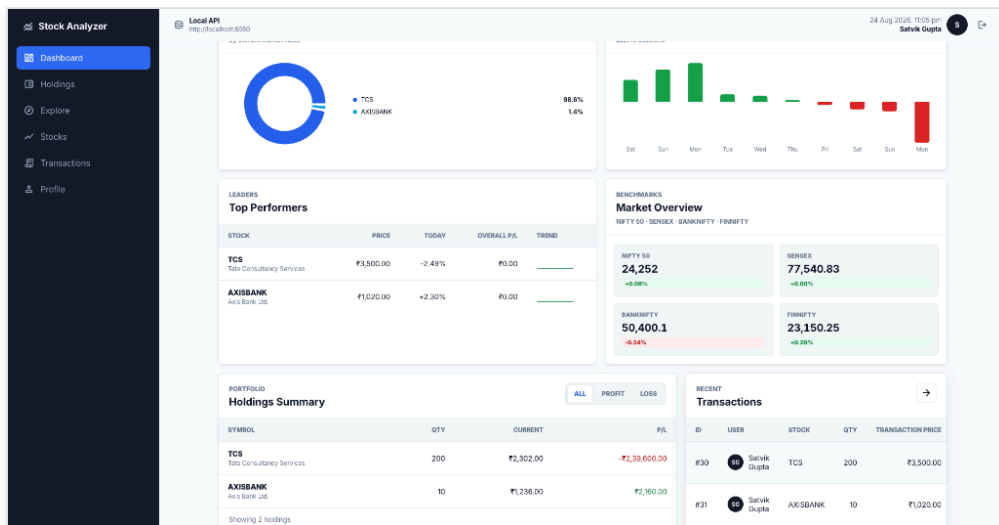


Figure 8.3: Portfolio Dashboard

The Dashboard provides a personalized, centralized overview of the user's financial standing. It dynamically displays Total Invested, Current Value, and Unrealized Profit/Loss. It also highlights active holdings and recent transaction activity, pulling data securely via the `/api/dashboard` endpoint.

8.3 Holdings

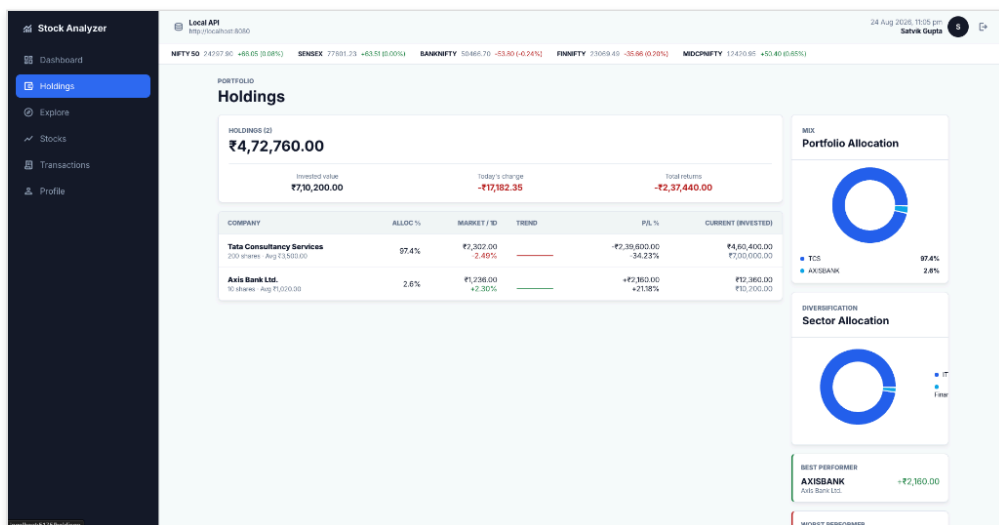


Figure 8.4: Holdings Interface

The Holdings interface presents a detailed breakdown of the user's active portfolio. For each stock owned, it calculates the allocated percentage, average cost basis, current market value, and individual profit/loss, giving the investor granular insight into asset performance.

8.4 Explore and Stocks

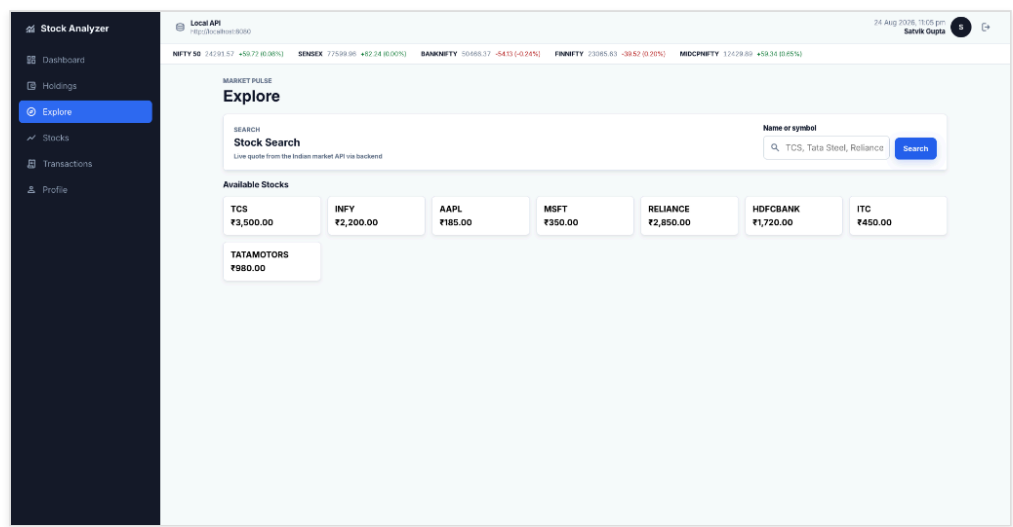


Figure 8.5: Stock Explorer

The Explore page allows users to browse the master catalogue of available stocks. It includes search functionality and displays real-time pricing information for equities available to trade within the application simulator.

8.5 Transaction History

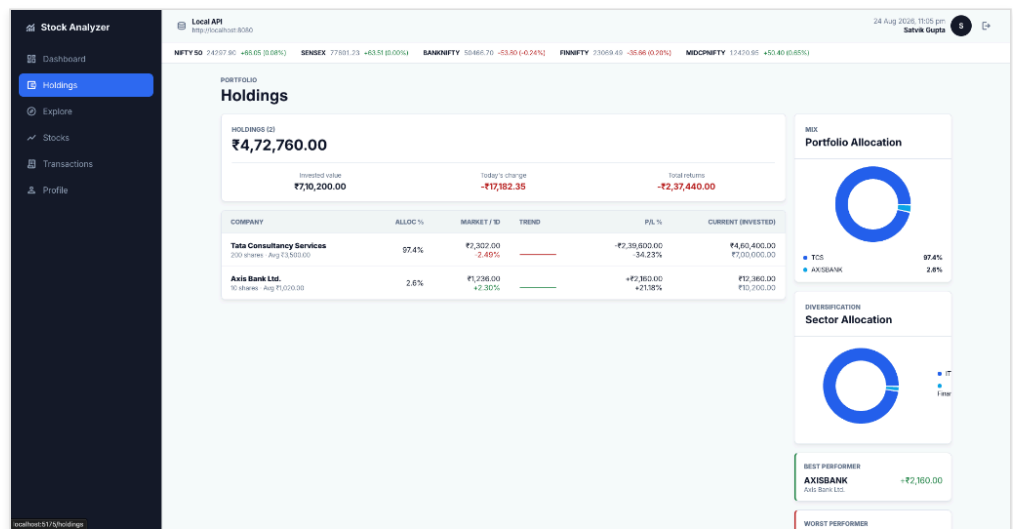


Figure 8.6: Transaction History

The Transactions page provides a comprehensive chronological ledger of all BUY and SELL activities performed by the user, detailing the date, asset, quantity, execution price, and total value of each trade.

8.6 Profile and Data Export

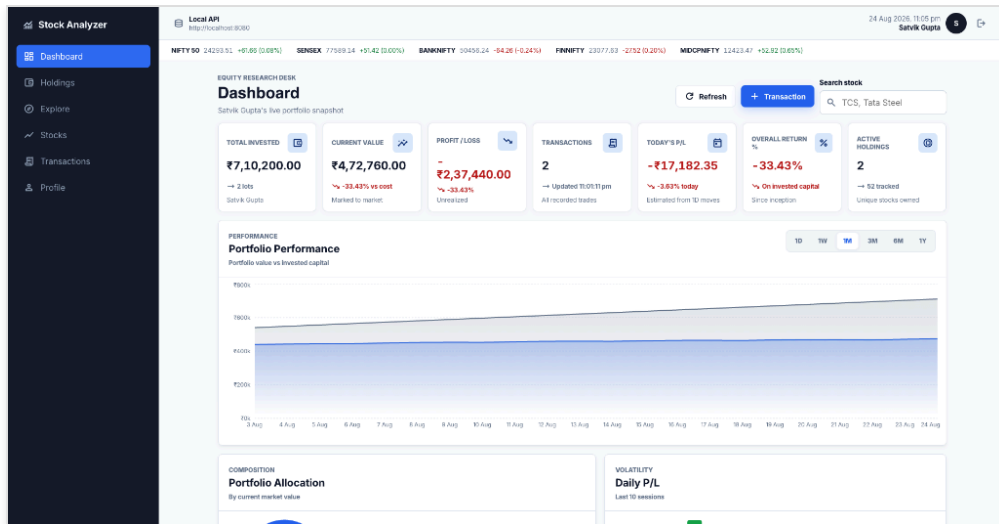


Figure 8.7: User Profile and Portfolio Summary

The Profile section acts as the user's account center. It displays authenticated personal details and a summary of their portfolio statistics. Crucially, it houses the **Data Management** controls, allowing users to export their transaction history as a CSV file or download a comprehensive, multi-sheet Excel portfolio report. It also includes the Logout functionality.

8.7 Change Password

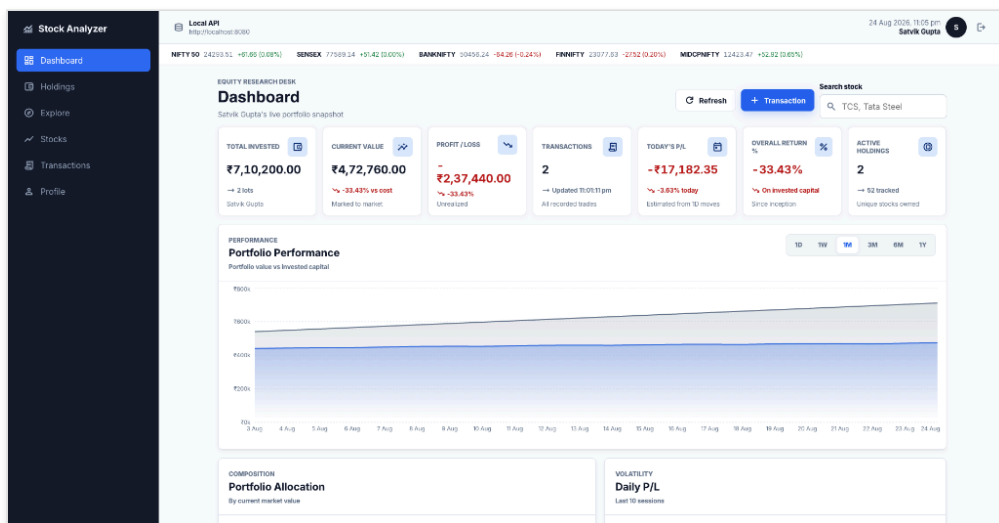


Figure 8.8: Change Password Interface

Accessible from the Profile page, the Change Password modal allows users to securely update their credentials. It requires the user to input their current password (verified via BCrypt on the backend) before a new password can be established, ensuring account security.

CHAPTER 9: TESTING

9.1 Testing Strategy

The application underwent rigorous testing encompassing static analysis, backend unit testing, API validation, and manual end-to-end (E2E) regression testing. The testing strategy focused heavily on verifying security boundaries, JWT authorization, business logic constraints, and data export fidelity.

9.2 Build Verification

Prior to runtime testing, static validation was performed:

- **Frontend Linting:** `npm run lint` executed successfully with 0 errors (with a known non-blocking `exhaustive-deps` warning).
- **Frontend Build:** `npm run build` successfully bundled the React application for production.
- **Backend Tests:** `./mvnw test` completed with 0 failures and 0 errors, confirming core Spring Boot context initialization.

9.3 Comprehensive Test Cases

Test ID	Feature	Test Case	Expected Result	Actual Result	Status
TC-01	Signup	Register with valid unique email & password.	200 OK, JWT returned, password not exposed.	Passed, JWT issued.	PASS
TC-02	Login	Login with correct credentials.	200 OK, JWT generated.	Passed, JWT issued.	PASS
TC-03	Login	Login with invalid password.	401 Unauthorized.	Rejected with 401.	PASS
TC-04	Security	Access <code>/api/dashboard</code> without JWT.	401 Unauthorized.	Rejected with 401.	PASS
TC-05	Security	Access <code>/api/dashboard</code> with valid JWT.	200 OK, User A data returned.	Passed, data loaded.	PASS
TC-06	Isolation	User B attempts to view User A's transactions.	Empty list or User B's data only.	User B sees 0 transactions.	PASS
TC-07	Transactions	Execute BUY order.	200 OK, Holdings increase, Invested value updates.	Passed, Dashboard updated.	PASS
TC-08	Transactions	Execute SELL order exceeding held quantity.	400 Bad Request, transaction rejected.	Rejected with 400.	PASS
TC-09	Profile	Update Profile Name.	200 OK, name persists in DB.	Passed, name updated.	PASS
TC-10	Security	Change Password with wrong current password.	401 Unauthorized.	Rejected with 401.	PASS
TC-11	Security	Change Password with correct credentials.	200 OK, password updated via BCrypt.	Passed.	PASS
TC-12	Security	Login with old password after change.	401 Unauthorized.	Rejected with 401.	PASS
TC-13	Export	Download CSV Export.	Valid CSV file with accurate transaction history.	Passed, file downloaded.	PASS
TC-14	Export	Download Excel Export.	Valid XLSX with Summary, Holdings, Transactions sheets.	Passed, 3 sheets verified.	PASS
TC-15	Routing	Access frontend <code>/dashboard</code> unauthenticated.	Redirect to <code>/login</code> .	Passed, redirected.	PASS

Table 9.1: Comprehensive Test Cases and Results

CHAPTER 10: ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

10.1 Advantages

The final implementation of the Stock Portfolio Analyzer provides significant advantages over manual tracking methods:

- **Robust Security:** Utilizing JWT for stateless authentication and BCrypt for password hashing ensures industry-standard data protection.
- **Strict Data Privacy:** The authenticated user ownership model rigorously isolates user data, preventing unauthorized access across accounts.
- **Automated Accuracy:** Real-time computation of holdings, average buy prices, and profit/loss eliminates human calculation errors.
- **Centralized Management:** A personalized Profile page consolidates personal info, portfolio summaries, and security settings in one accessible location.
- **Seamless Data Portability:** Automated CSV and multi-sheet Excel exports allow users to effortlessly download their financial records.
- **Scalable Architecture:** The strict separation of concerns across the React frontend and layered Spring Boot backend ensures the system is easily maintainable and extensible.

10.2 Limitations

While the core system is fully functional, certain limitations exist in the current version:

- **Static Market Data:** Stock prices currently rely on manual database updates rather than a live, real-time market data feed API.
- **Local Deployment:** The application is currently configured for a local development environment and lacks cloud-native deployment configurations.
- **Limited Analytics:** The application provides tabular data and summary metrics but currently lacks advanced graphical charts (e.g., line charts for performance over time).
- **No Automated Alerting:** Push notifications, SMS, or email alerts for significant portfolio changes are not yet integrated.

10.3 Future Scope

The modular architecture provides a strong foundation for future enhancements:

- **Real-Time API Integration:** Connecting to a financial market API (e.g., Alpha Vantage, Yahoo Finance) for live price updates.
- **Advanced Visual Analytics:** Implementing charting libraries (like Chart.js or Recharts) to provide visual representations of sector diversification and historical performance.
- **AI-Driven Recommendations:** Integrating predictive analytics to suggest portfolio rebalancing or highlight potential investment opportunities.
- **Cloud Deployment:** Containerizing the application with Docker and deploying it to cloud platforms like AWS or Heroku.
- **Mobile Application:** Developing a companion mobile application (e.g., using React Native) that consumes the existing REST APIs.

CHAPTER 11: CONCLUSION

The Stock Portfolio Analyzer project has successfully demonstrated the design, development, and implementation of a secure, full-stack, three-tier web application that addresses the inefficiency and error-proneness of manual portfolio management. By combining a modern React.js presentation layer, a robust Spring Boot business-logic layer, and a structured MySQL data layer, the project produced a highly cohesive system.

A major achievement of the final implementation is its rigorous security posture. The integration of JWT authentication, BCrypt password hashing, and strict authenticated-user ownership checks ensures that financial data remains private and protected. The layered Controller–Service–Repository–Entity pattern allowed development effort to focus on complex business logic, such as automated portfolio valuation, transaction validation, and the generation of dynamic CSV and Excel data exports.

Systematic end-to-end testing confirmed the functional correctness of the core flows and the impenetrability of the security boundaries. The project stands as a highly practical demonstration of modern full-stack technologies applied to a tangible personal-finance problem, providing a solid foundation for continued learning in financial technology, web security, and software engineering.

REFERENCES

- [1] Spring Boot Official Documentation. Available at: <https://docs.spring.io/spring-boot/documentation.html>
- [2] Oracle Java SE Documentation. Available at: <https://docs.oracle.com/en/java/javase/>
- [3] MySQL 8.0 Reference Manual. Available at: <https://dev.mysql.com/doc/>
- [4] Hibernate ORM User Guide. Available at: <https://hibernate.org/orm/documentation/>
- [5] React Official Documentation. Available at: <https://react.dev/>
- [6] JSON Web Token (JWT) Introduction. Available at: <https://jwt.io/introduction>
- [7] Spring Security Reference. Available at: <https://docs.spring.io/spring-security/reference/index.html>
- [8] Apache POI - the Java API for Microsoft Documents. Available at: <https://poi.apache.org/>
- [9] REST API Design Resources. Available at: <https://restfulapi.net/>
- [10] Spring Data JPA Reference Documentation. Available at: <https://docs.spring.io/spring-data/jpa/reference/>

APPENDIX

A.1 List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
BCrypt	Blowfish Cryptographic Hashing Algorithm
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
REST	Representational State Transfer
RDBMS	Relational Database Management System
SPA	Single-Page Application
SQL	Structured Query Language
UI / UX	User Interface / User Experience

Table A.1: List of Abbreviations